

¿Qué es el patrón Singleton?

El **patrón Singleton** (en español, *único*) es un **patrón de diseño** que se usa cuando **queremos que solo exista un único objeto** de una clase en toda la aplicación.

💡 Es decir:

Solo se puede crear **una instancia** (un único objeto) y, si intentamos crear más, siempre nos devolverá la **misma**.

🤖 ¿Por qué puede ser útil?

Imagina que en tu aplicación web tienes un **objeto de configuración** (por ejemplo, con la URL del servidor, el idioma, o la versión de la app).

No tendría sentido que cada parte del programa tuviera su propia copia: queremos **una sola configuración compartida**.

Cómo se haría sin el patrón Singleton

```
class Configuracion {  
  constructor() {  
    this.urlServidor = "https://api.midominio.com";  
    this.idioma = "es";  
  }  
}
```

```
const config1 = new Configuracion();  
const config2 = new Configuracion();
```

```
console.log(config1 === config2); // ✗ false → Son objetos diferentes
```

Aquí estamos creando **dos objetos distintos**.

Aunque tengan el mismo contenido, **no son el mismo objeto**.

Aplicando el patrón Singleton

Ahora lo transformamos para que **solo haya una única instancia posible**:

```
class Configuracion {  
  // Guardamos la instancia en una propiedad estática de la clase  
  static instancia;  
  
  constructor() {  
    // Si ya existe una instancia, devolvemos la misma  
    if (Configuracion.instancia) {  
      return Configuracion.instancia;  
    }  
  
    // Si no existe, creamos una nueva  
    this.urlServidor = "https://api.midominio.com";  
    this.idioma = "es";  
  
    // Guardamos la instancia recién creada  
    Configuracion.instancia = this;  
  }  
}  
  
// Probamos  
const config1 = new Configuracion();  
const config2 = new Configuracion();  
  
console.log(config1 === config2); // ☒ true → Es el mismo objeto
```

Explicación paso a paso

1. **static instancia;**
→ Propiedad estática que pertenece a la clase (no a los objetos).
Aquí guardaremos la única instancia.
 2. **En el constructor:**
 - Si **ya existe una instancia**, la devolvemos.
 - Si **no existe**, la creamos y la guardamos.
 3. De esta forma, **aunque usemos new muchas veces**, solo tendremos **un único objeto compartido**.
-

Ejemplo práctico en una web

Podríamos usar este patrón para gestionar un **carrito de compra** que debe ser único en toda la aplicación:

```
class Carrito {
  static instancia;

  constructor() {
    if (Carrito.instancia) {
      return Carrito.instancia;
    }

    this.productos = [];
    Carrito.instancia = this;
  }

  agregarProducto(producto) {
    this.productos.push(producto);
  }

  mostrarProductos() {
    console.log(this.productos);
  }
}

const carrito1 = new Carrito();
const carrito2 = new Carrito();

carrito1.agregarProducto("Teclado");
carrito2.agregarProducto("Ratón");

carrito1.mostrarProductos(); // ["Teclado", "Ratón"]
carrito2.mostrarProductos(); // ["Teclado", "Ratón"]

console.log(carrito1 === carrito2); // true ☒
```

Aquí, tanto carrito1 como carrito2 son **exactamente el mismo objeto**, compartiendo los productos.

En resumen

Concepto	Explicación
Qué hace	Garantiza que solo haya un objeto de una clase.
Dónde se usa	Configuración global, conexión a base de datos, gestor de logs, etc.
Ventajas	Evita duplicados, mantiene datos coherentes.
Inconvenientes	Puede acoplar demasiado el código si se abusa.